

5.3 Testing Approach

Testing is an essential part of the development lifecycle of the Smart Event Management and Ticket Booking System, ensuring it functions correctly, meets the required specifications and integrates seamlessly with all other elements of the application, such as the user interface, back-end services, APIs and database. Full testing carried out throughout to identify and fix issues on both a module and an application level.

Assuming various testing techniques, such as Unit Testing, Integration Testing and System Testing. User Management and Authentication, Event Management, Ticket Booking, Payment Processing, Database Operations, API Communication, and Administrative functionalities were given special attention. The systematic review enabled identifying and rectifying any potential issues before deployment, enhancing system reliability, security, and overall user experience.

5.3.1 Unit Testing

The individual components and functions of the system were tested with unit testing to ensure they functioned properly and independently. All modules were individually tested so that they behave as it is expected to do without reliance upon other parts of the system.

As part of the testing, it was addressed, among other important things, the logics of user registration, initiating a user's login, generating a JWT authentication token, user verification by user role and also the user's password verification via the bcrypt encryption technique. Event related data processing, deployment of booking calculations, payment calculations, handling of api requests, database connection and validation of the data input in forms were also checked.

For instance, tests were conducted on the registration feature with both valid and invalid entered, including missing data, incorrect email format, duplicate email addresses, etc. In the same way, login module was tested with both valid and invalid credentials to ensure that customers with correct credentials were able to access the system with correct credentials, and customers with incorrect credentials were unable to login to the system.

Early discovery of defects via the unit tests has prevented some of the incidents from occurring when the various parts were connected into the full application. This helped to enhance the reliability, accuracy and quality of the system.

5.3.2 Integration Testing

Integration testing was performed to ensure the correct communication and functioning of various modules and technologies in the system. Major consideration was on the smooth interaction between the front end, back end, database, authentication services, and administration functions.

The following integrations were tried:

Here we will explore how Express and React communicate. Let's look at how Express communicates with React.

Connect Express back end to MongoDB Atlas.

Encapsulation of authentication API with front end operations.

Protect routes: authentication and authorization. Protected routes: authentication and authorization.

The database activity that supports event management operations. Entities of the kind that support operations of event management in the database.

Managing bookings with event related information.

Payment through bookings details.

Two-way communication between the administrative back-end and APIs.

A user signed up from the React frontend and passed the data on to the Express backend via an API call, for instance. The server-side checked this info and then securely encrypted the supplied password using bcrypt, adding it to the MongoDB Atlas database with the user data. All these stages were validated for their processing and transfer of the data.

Methods of authentication and consequently, token validation were also studied to ensure that only those with security clearance were able to find and read certain areas, and that unauthorized ones were not allowed to go there. Such tests had to be implemented to ensure optimal cooperation between the front end and back end processes, yet included the necessary security precautions.

5.3.3 System Testing

The entire Smart Event Management and Ticket Booking System was put through a system testing to ensure the functionality of all the components as a single application. Major workflows and features for both user and administrator were covered in the testing.

The following features were evaluated: user registration and log in, administrator authentication, event browsing, event management, ticket booking, payment processing and booking status in real-time. In addition to this, the user dashboard, safe API interactions, secured routes and error handlers were checked. Furthermore, the attunement of user interface and the deployment procedure were tested.

The app was further tested in the real environment to ensure that the Vercel FrontEnd, Railway BackEnd and MongoDB Atlas Database work together. This assisted to verify the correct operations of the deployed system in real operating conditions.

System testing results show that the major functionalities worked as intended. Any issues detected during this testing process were discussed and worked out prior to deployment. As such, the testing process has given more importance about the capability, dependability, security and usability of the system.

